

Wilhelm

AI Prompts, System Workflow & Development Log

KOHLER Hackathon — Track 3: Unified Enterprise AI Agent

1. Methodology

Wilhelm was built using an AI-assisted, spec-driven development workflow through Kiro. Rather than issuing large, multi-part instructions, the build was broken into a sequence of small, single-task prompts — one clearly scoped task per prompt, each followed by its own commit. This kept every AI-generated change reviewable in isolation and every commit a working checkpoint the project could be demoed from at any point.

The system was designed to be provider-agnostic from the start: all LLM calls and all embedding/vector-store calls are routed through internal adapter modules (see Prompts 3 and 4 below), rather than calling any provider SDK directly throughout the codebase. This allows the same application code to run against OpenAI, Gemini, or a locally hosted model (Ollama, LM Studio, vLLM, or any OpenAI-compatible local server) purely via environment variable configuration.

A core architectural principle enforced throughout the prompts below: the assistant's answers are required to be grounded in retrieved knowledge-base content rather than generated from the model's general training knowledge, with an explicit instruction for the model to state when information is not available rather than fabricate an answer.

2. Architecture Summary

Stage	Responsibility
Router	Classifies each incoming query into a domain (hr / support / out_of_scope) via the LLM adapter.
Retrieval	Embeds the query and retrieves the top-k relevant chunks from the vector store, scoped to the routed domain.
Grounded Answering	Generates an answer using only the retrieved context, with an explicit instruction to decline rather than fabricate when context is insufficient.
Format Dispatcher	Reformats the grounded answer on demand into JSON, XML, an Excel file, or a draft email, each with a validation parse-check.
Conversation Memory	Maintains session-level turn history so follow-up requests (e.g. "now give me that as JSON") resolve against prior context.

3. Full Prompt & Commit Log

Every prompt below was issued to Kiro as a single, scoped task. Each was verified working before the corresponding commit was made.

Phase 0 — Project Setup

Prompt 1: Scaffold a new project: React + Vite frontend, Node (or Python/FastAPI) backend, basic folder structure for frontend/ and backend/. No features yet, just the skeleton and a working dev server on both sides.

`Commit: Set up initial project scaffold (frontend + backend)`

Prompt 2: Add environment config — .env.example with an LLM_PROVIDER variable (openai / gemini / local), an LLM_BASE_URL variable (used only when LLM_PROVIDER=local, e.g. http://localhost:11434/v1 for Ollama or http://localhost:1234/v1 for LM Studio), and an EMBEDDING_PROVIDER variable, plus placeholder keys for each. Install core dependencies for whichever providers you plan to support, but no logic yet.

`Commit: Set up environment config and core dependencies`

Prompt 3: Write an LLM adapter module with a single exported function generateText(prompt, options) that switches on LLM_PROVIDER: openai/gemini call their respective SDKs, and local calls any OpenAI-compatible server (Ollama, LM Studio, vLLM, llama.cpp) at LLM_BASE_URL using the OpenAI SDK pointed at that base URL — no API key required for local. All three paths return a plain string. The rest of the app should only ever call generateText(), never a provider SDK directly.

`Commit: Add provider-agnostic LLM adapter (cloud + local)`

Prompt 4: Write an embedding + vector-store adapter with three exported functions — embedText(text), upsertVectors(items), and queryVectorStore(embedding, topK, filter) — that internally switch on EMBEDDING_PROVIDER/VECTOR_STORE_PROVIDER (e.g. local sentence-transformers + Chroma, or OpenAI embeddings + Pinecone). The rest of the app should only call these three functions, never a specific SDK.

`Commit: Add provider-agnostic embedding/vector-store adapter`

Phase 1 — Knowledge Base Layer

Prompt 5: Write 10–15 short, realistic HR policy documents (plain text/markdown) covering topics like leave policy, remote work, expense reimbursement, onboarding. These are seed data, not code.

`Commit: Add seed HR knowledge base documents`

Prompt 6: Write 10–15 short IT/support documents (password reset, VPN access, hardware requests, ticket escalation process). Seed data only.

`Commit: Add seed IT/support knowledge base documents`

Prompt 7: Write a chunking script that splits documents into overlapping text chunks (e.g. 200–300 words, small overlap) and outputs them as a list with source metadata (domain, filename).

`Commit: Add document chunking script`

Prompt 8: Write an ingestion script that runs chunking on all seed documents, calls embedText() from the adapter on each chunk, and stores the results via the adapter's vector-store write function with metadata attached.

`Commit: Add knowledge base ingestion script`

Phase 2 — Router Agent

Prompt 9: Write a router function that takes a user query, calls `generateText()` from the LLM adapter with a classification-only prompt, and returns a domain label (hr / support / out_of_scope).

`Commit: Add query router / domain classifier`

Prompt 10: Write 8–10 test queries (mix of clear HR, clear support, ambiguous, and clearly out-of-scope) and a small script that runs them through the router and prints the results for manual review.

`Commit: Add router classification smoke tests`

Phase 3 — Retrieval + Grounded Answering

Prompt 11: Write a retrieval function that takes a query and domain, calls `embedText()` and `queryVectorStore()` from the adapter, and returns the top-k most relevant chunks for that domain only.

`Commit: Add domain-scoped retrieval function`

Prompt 12: Write an answer-generation function that takes the query and retrieved chunks, and calls `generateText()` with a prompt instructing the LLM to answer only using the provided context, explicitly told to say “I don’t have that information” if the context doesn’t cover it.

`Commit: Add grounded answer generation with hallucination guard`

Prompt 13: Wire router → retrieval → answer generation into one pipeline function. For out_of_scope domain, skip retrieval and return a fixed fallback message instead.

`Commit: Add end-to-end query pipeline (router + RAG + fallback)`

Phase 4 — Format Agent

Prompt 14: Write a formatter function that takes a raw answer and reformats it as JSON matching a simple schema (e.g. `{ "answer": str, "domain": str, "sources": [str] }`). Validate the output with a JSON parse check before returning it.

`Commit: Add JSON output formatter with validation`

Prompt 15: Add an XML formatter for the same answer structure, with a parse-check validation step.

`Commit: Add XML output formatter with validation`

Prompt 16: Add an Excel formatter: takes the answer (or a batch of Q&A; pairs) and generates a downloadable .xlsx file using a spreadsheet library.

`Commit: Add Excel output formatter`

Prompt 17: Add an email-draft formatter: takes the answer and generates a subject line + body in a professional email format, returned as plain text.

`Commit: Add email draft formatter`

Prompt 18: Write a single dispatcher function that takes a format name (json/xml/xlsx/email) and routes to the correct formatter. No new logic, just the switch/dispatch.

`Commit: Add format dispatcher`

Phase 5 — Conversation Memory

Prompt 19: Add a simple in-memory conversation history structure (array of turns) per session, and pass relevant prior turns into the pipeline so follow-up queries like “now give me that as JSON” have context.

`Commit: Add session conversation memory`

Phase 6 — Frontend

Prompt 20: Build a basic chat UI skeleton: message list, input box, send button. No backend wiring yet, just local state with dummy messages.

`Commit: Add chat UI skeleton`

Prompt 21: Wire the chat UI to the backend pipeline endpoint — send user message, display returned answer as plain text.

`Commit: Wire chat UI to backend pipeline`

Prompt 22: Add a format selector (buttons or dropdown: plain / JSON / XML / Excel / email) next to the chat input, and pass the selected format to the backend.

`Commit: Add output format selector in chat UI`

Prompt 23: Update the chat UI to render each format correctly — code block for JSON/XML, download link for Excel, styled draft box for email.

`Commit: Add format-aware response rendering`

Phase 7 — UI Polish: Dispatch Console Design

Prompt 24: Add Google Fonts “Space Grotesk” (headings/UI labels) and “JetBrains Mono” (only for real data — JSON/XML previews), and set up a CSS variable token system: background #0B1120, panel #131B2E, border #243149, text primary #E7ECF5, text muted #8A96AD, accent-routing #5EEAD4 (teal), accent-sourced #F5A623 (amber). Also define a light-mode override set of the same tokens for prefers-color-scheme: light. No layout changes yet, just tokens and fonts wired in.

`Commit: Add design token system and fonts`

Prompt 25: Build the 3-zone desktop layout shell using CSS grid: a left department rail (~240px), a center conversation panel (flexible width), and a right trace panel (~280px). Use the token system from the previous step, near-zero border-radius, hairline borders — no cards-with-shadow styling. No content inside the zones yet, just the shell.

`Commit: Add 3-zone dispatch console layout shell`

Prompt 26: Build the department rail: list HR and IT/Support as active items (icon + label), and Finance and Legal as locked items (dimmed, with a lock icon) to honestly reflect current domain scope. Add a highlighted/glowing state for whichever department the last query was routed to.

`Commit: Add department rail with active and locked states`

Prompt 27: Build the trace panel as 4 static nodes in a vertical list: Router, Retrieval, Grounding, Format, each with a label and an idle/pending visual state (dim, unfilled). No animation yet, just the static structure.

`Commit: Add static trace panel nodes`

Prompt 28: Wire the trace panel to animate when a query is sent: each node lights up in sequence (Router →

Retrieval → Grounding → Format) with a short connecting-line pulse between them, timed to roughly match how long each real backend step takes. This is the one deliberately bold animated moment in the UI — keep every other interaction (hover states, etc.) minimal by contrast.

`Commit: Animate trace panel sequence on query submit`

Prompt 29: Add a source-grounding tag under each assistant answer (amber, using the accent-sourced token) showing which knowledge base documents were retrieved, expandable on click to show the retrieved snippet text.

Commit: Add expandable source attribution tag

Prompt 30: Build the format switcher as a segmented control (Plain / JSON / XML / Excel / Email) above the chat input. Selecting a format re-renders the last assistant answer in that format, using the monospace font for JSON/XML previews and a distinct styled block for the email draft preview.

Commit: Add format switcher with per-format rendering

Prompt 31: Add an empty state for a fresh session: a short prompt line plus 2–3 clickable example query chips (auto-filling the input) so the demo has an obvious starting point instead of a blank console.

Commit: Add empty state with example query chips

Prompt 32: Make the layout responsive: collapse the department rail and trace panel into toggle drawers (buttons that slide the panel in/out) on narrow viewports, keeping the conversation panel as the primary view on mobile.

Commit: Make dispatch console responsive with collapsible drawers

Prompt 33: Accessibility and motion pass: visible keyboard focus states on all interactive elements, prefers-reduced-motion fallback that disables the trace animation pulse (nodes still update state, just without the moving pulse), and confirm text contrast against both dark and light token sets.

Commit: Add accessibility and reduced-motion support

Phase 8 — Final Polish

Prompt 34: Write a short demo script: 4–5 prepared example interactions (including a multi-turn one — ask a question, then ask for it reformatted) to use for the live demo.

Commit: Write demo script

Prompt 35: Write the README: what Wilhelm does, architecture summary, how to run it locally, known limitations (e.g. only 2 domains covered). Include an explicit line disclosing that the HR/IT knowledge base documents are synthetic demo data generated for this hackathon, not real KOHLER policy, and confirm the seed documents are committed to the repository for reproducibility.

Commit: Write README

Prompt 36: Add basic error handling: empty query, LLM API failure, vector store empty — each should fail gracefully with a user-facing message, not a crash.

Commit: Add graceful error handling for edge cases

4. Workflow Notes

- Each Kiro prompt was scoped to a single task — no bundled or multi-part asks — so every step was individually reviewable and every commit represented a clean, working checkpoint.
- Commits were made only after a prompt's output was verified working, not immediately upon generation.
- Every LLM and embedding/vector-store call in the application routes through the adapters built in Prompts 3 and 4, never a provider SDK directly — allowing the LLM provider (OpenAI / Gemini / any local OpenAI-compatible server such as Ollama, LM Studio, or vLLM) and the vector store (Chroma / Pinecone) to be swapped via environment configuration without touching downstream application code.
- UI polish (Phase 7) was deliberately sequenced after core pipeline functionality (Phases 0–6) was verified working, so visual work never came at the expense of the underlying grounding/retrieval correctness.
- The knowledge base scope was intentionally limited to two domains (HR and IT/Support) with full depth, rather than shallow coverage across all five domains named in the original brief, to keep the demo's grounding claims verifiable.